

Repository access

Pull the project from the canonical GitHub repository:

`https://github.com/Minkeejung0415/rehab-robotics-studio.git`

New workstation

```
git clone https://github.com/Minkeejung0415/rehab-robotics-studio.git
cd rehab-robotics-studio
```

Existing checkout

```
git status --short
git pull origin master
```

Commit or safely stash intentional local changes before pulling. Do not overwrite local work.

Sensor signal	Master: firmware/step_node/step_node.ino	Slave: firmware/step_node_slave/step_node_slave.ino
INT / DIO	PIN_DIO = D0 (GPIO1)	PIN_DIO = D0 (GPIO1)
Power	board-specific 3.3 V and GND	board-specific 3.3 V and GND

The Master sketch contains nearby diagnostics/comments that do not always agree with its compiled pin defines. Use the `#define PIN_*` lines as the source authority, then boot the board and confirm the IMU WHO_AM_I check before trusting streamed data. Update code and recompile if the harness differs; do not update only a comment.

Both DIO inputs use INPUT_PULLUP and ISR capture on both signal edges. Connect one clean, 3.3 V-compatible trigger output to both D0/GPIO1 inputs and connect trigger ground to both nodes. First test one edge at a low rate, then verify both streams have matching edges before moving to 500 Hz.

4. microSD, radio, and network prerequisites

Each sketch calls `SD.begin(PIN_SD_CS, SPI, 25000000)` with `PIN_SD_CS = 21`. The project defines only the chip-select pin explicitly; verify the remaining SPI routing against the exact XIAO board/core and microSD breakout before finalizing a harness.

Master/Slave use ESP-NOW and currently compile `ESPNOW_WIFI_CHANNEL` as 6; the active Wi-Fi/channel topology must be compatible. TCP control is port 5000; live UDP is port 55001. Keep this traffic on a trusted network and never place credentials in documents or source control.

5. Jetson deployment choices

Topology	Use it when	Data path
Existing Windows + WSL	preserving the tested desktop workflow	ESP32 -> UDP 55001 -> Windows relay -> WSL ROS 2 -> rosbridge 9090 -> Studio
Direct Jetson	deploying a standalone Linux acquisition host	ESP32 -> UDP 55001 -> Jetson ROS 2 bridge -> rosbridge 9090 -> Studio
Jetson auxiliary sensors	adding EMG/load cells while retaining ESP IMUs	ESP and Jetson nodes -> common ROS graph -> Studio

For direct Jetson acquisition, join Jetson and both ESP32 nodes to the same trusted 2.4 GHz Wi-Fi/hotspot. Jetson opens TCP control to each node on 5000, binds one UDP receiver on 55001, and identifies devices by canonical full MAC/source metadata. ESP-NOW remains the Master-to-Slave synchronization path, not a Jetson control bus.

Install ROS 2 Humble, `rosbridge_websocket`, and each vendor acquisition driver. If Studio is remote, expose `rosbridge` on 9090 only through the correct firewall and ROS domain configuration. Validate the exact current 50-byte little-endian ESP UDP frame before porting a bridge; do not change its field order or byte order.

6. Adding EMG and load cells safely

Do not insert EMG or load-cell data into the fixed ESP IMU packet. Each acquisition board should have a Jetson ROS 2 node that publishes `device_id`, source `timestamp_ns`, sequence, configured/observed rate, channel names, values, and health/drop status.

```
/sensors/emg/<device-id>/raw
/sensors/emg/<device-id>/envelope
/sensors/load_cell/<device-id>/raw
/sensors/load_cell/<device-id>/calibrated
/sensors/<device-id>/status
```

Keep EMG raw data separately from filtered/rectified/envelope values. Keep load-cell ADC counts as well as tare/calibration-transformed force or torque. Record full-rate data on Jetson/ROS bag; send decimated views to Studio. The exact electrodes, amplifier, ADC, bridge, and excitation wiring must come from the selected vendor hardware; this repository does not contain sufficient information to invent those connections.

7. Setup acceptance checklist

- Both boards boot and each IMU passes WHO_AM_I.
- DIO transitions are seen by both nodes at low rate.
- SD card performs a normal short record/stop cycle.
- Master/Slave pair and canonical IDs appear in ROS registry.
- Studio connects through rosbridge and receives the expected streams.
- If Jetson is used, its ROS topics agree with the existing bridge for the same device.
- Record the exact wiring revision, firmware version, MAC IDs, network topology, and test result in the handoff record.

8. Flash the firmware and perform USB commissioning

The firmware is self-contained in two sketch folders. Each folder contains its matching `vqf.c` and `vqf.h`; do not upload only the `.ino` file and omit these companion files.

Role	Sketch to open/upload	Current firmware identifier	Expected sensor evidence
Master	firmware/step_node/step_node.ino	FIRMWARE_VERSION "1.8.0"	ICM20948: OK ... WHO_AM_I=0xEA
Slave	firmware/step_node_slave/step_node_slave.ino	FIRMWARE_VERSION "1.8.0"	ICM20948: OK ... WHO_AM_I=0xEA

8.1 One-time Arduino environment procedure

- Install Arduino IDE and an ESP32 board package that exposes the official Seeed XIAO ESP32S3 target. The firmware deliberately checks for the compile symbol `ARDUINO_XIAO_ESP32S3`; record the exact package/version that produces it in the commissioning record.
- Connect one board by USB-C. In Arduino IDE select the XIAO ESP32S3 target and the COM port shown by Windows. The normal diagnostics/test scripts use 115200 baud.
- Open the Master sketch for the Master-labelled physical board. Review the network defines without copying credentials into a public document. Confirm the ESP-NOW and active Wi-Fi channel plan agree with the lab setup.
- Compile and upload. Repeat separately with the Slave sketch and the Slave-labelled physical board.
- Open serial output at 115200. Record the complete boot output, board role, canonical identity response, WHO_AM_I, SD readiness, Wi-Fi association/AP status, and DIO initial level.
- If WHO_AM_I is not 0xEA, stop. Check 3.3 V, common ground, the role-specific SPI table, CS wiring, and physical board/module identity before continuing. The firmware can announce a synthetic fallback; that is a diagnostic failure, not permission to treat the IMU as verified.

The repository does not pin an Arduino IDE version, ESP32 board-package version, or USB driver version. The first successful commissioning run must freeze those versions in 04-COMMISSIONING-RECORD.md so a future reinstall is reproducible.

8.2 USB-only DIO smoke test

With both boards flashed and visible as separate Windows ports, run this test before debugging Wi-Fi:

```
python scripts#test_dio_sync_serial.py --master-port COM3 --slave-port COM4 --duration 60 --sample-rate 500 --threshold-ms 2
```

Replace COM3/COM4 with the actual labelled ports. The script opens both at 115200, asks both boards to run at 500 Hz, enables DIO monitoring, pairs edges, checks the configured rate, and fails if maximum paired skew

exceeds the supplied threshold. Save its raw console output with the date and firmware hash. The short ISR smoke result passed previously; the 900-second strict test remains open and is documented in the Project Map guide.

9. What must be captured on site before this becomes fully reproducible

The repository alone cannot prove the exact physical assembly. Complete the companion 04-COMMISSIONING-RECORD.md once for each lab build. It is not optional paperwork: without its evidence a new maintainer cannot safely reproduce the system.

At minimum attach or link the following controlled artifacts:

- top, bottom, and connector-side photographs of every Master/Slave assembly, with visible role labels;
- exact manufacturer, part number, board revision, supplier link, and quantity for XIAO, IMU breakout, microSD breakout/card, trigger generator, cables, power source, Jetson, EMG equipment, and load-cell/ADC equipment;
- a labelled wiring photograph and a continuity-checked pin-to-pin table for each harness;
- an export of the Arduino IDE/board package/library/USB driver versions that compiled the successful firmware;
- a secret-manager reference (not the secret itself) for Wi-Fi access, plus SSID, channel, addressing mode, and firewall rules;
- baseline serial logs, ROS topic samples, Studio screenshots, an SD sample file, and DIO test output;
- Jetson image, Ubuntu/ROS/driver version, MAC/IP reservation policy, and calibration-file location where Jetson is used.